

Dhirubhai Ambani University

(Formerly known as DA-IICT)

Topic: Datatypes in Python

Dr. Ankit Vijayvargiya

Assistant Professor

Room No. 4205, Faculty Block 4

Email: `ankit_Vijayvargiya[at]dau.ac.in`

Datatypes in Python

- Numeric
 - Integer
 - Float
 - Complex number
- Text
 - string
- Boolean
- None
- sequence
 - List
 - Tuple
 - range
- Set
 - Set
 - frozenset
- Mapping
 - Dictionary

Data types: Number/Numeric

Number data types store numeric values. They are immutable data types, which means that changing the value of a number data type results in a newly allocated object.

```
: x = 5
  print(id(x))
  # Let's print the memory address of x

  x = x + 1
  print(id(x))
  # Memory address changes because integers are immutable

140732584805288
140732584805320
```

Integers (default for numbers)

$Z=5$

Floats

$Z=5.1$

How to check the datatype:

```
: X=2
  print(type(X))
  <class 'int'>

: y=3.2
  print(type(y))
  <class 'float'>

: z=3+4j
  print(type(z))
  <class 'complex'>
```

Complex Number

$Z=5+2j$

Data types: Sequence

Strings:

- ❖ A string is a collection of one or more characters put in a single quote, double quote, or triple quote.
- ❖ Immutable datatype
- ❖ In Python, there is no character data type; a character is a string of length one.

`Z = 'python'`

`Z = "python"`

`Z = """python
programming
is very important"""`

Data types: Sequence

Accessing the character in Strings:

```
Text="python"  
012345
```

```
print(Text[0])
```

Output- p

```
Print(Text[0:1])
```

Output- p

```
print(Text[-3])
```

Output????

String Operations:

```
S1="Ankit"
```

```
S2="Vijay"
```

Concatenate:

```
S3=S1+S2
```

```
print(S3)
```

Output- Ankit Vijay

Repetition:

```
S3=S1*3
```

```
print(S3)
```

Output- Ankit Ankit Ankit

Membership test:

```
print("An" in S1) O/p- True
```

```
print("Vi" in S1) O/p-False
```

Data types: Sequence

Some useful string methods:

upper()

```
a="ankit"  
b=a.upper()  
print(b)
```

ANKIT

lower()

```
a="ANKIT"  
print(a.lower())
```

ankit

strip() –lstrip() rstrip()

```
a="    Ankit vijay    "  
print(a.strip())
```

Ankit vijay

replace()

```
a="Ankit Vijayvargiya"  
b=a.replace("Vijayvargiya", "vijay")  
print(b)
```

Ankit vijay

```
a="banana"  
b=a.replace("ana","vij")  
b  
  
'bvijna'
```

split()

```
a="Ankit Vijay Vargiya"  
b=a.split()  
print(b)
```

['Ankit', 'Vijay', 'Vargiya']

join()

```
a=["Ankit","Vijay","Vargiya"]  
b="-".join(a)  
print(b)
```

Ankit-Vijay-Vargiya

Checking string content

```
text = "Python123"  
  
print(text.isalpha())    # False  
print(text.isdigit())    # False  
print(text.isalnum())    # True
```

```
text = "PYTHON123"  
print(text.isupper())    # True
```

True

```
text = "python123"  
print(text.islower())    # True
```

True

```
: text = "PYTHON123"  
  
if text.isupper() and text.isalpha():  
    print("Only uppercase letters")  
else:  
    print("Not only uppercase")
```

Not only uppercase

Data types: Sequence

String Formatting:

It is the process of inserting variables or values into a string in a readable and controlled way.

Name= "Ankit"

Age=20

I want to print –My name is Ankit and I am 20 years old.

```
Name="Ankit"  
age=20  
print("My name is",Name,"and I am", age, "years old")
```

My name is Ankit and I am 20 years old

Using % formatting (early python)

```
print("My name is %s and I am %d years old" % (Name,age))
```

My name is Ankit and I am 20 years old

Using format (python 2.7+)

```
print("My name is {} and I am {} years old".format(Name,age))
```

My name is Ankit and I am 20 years old

Using f-string (python 3.6+)

```
print(f"My name is {Name} and I am {age} years old")
```

My name is Ankit and I am 20 years old

Data types: Boolean

It represents a logical value and has only two possible values:

- True
- False

Boolean values are often the result of a comparison operator.

x=10

Y=5

print(x>y) o/p???

print(x==y) o/p???

print((x>y)+(x==y))

o/p- 1

print((x>5)+(x>6))

o/p- 2

Data types: None

None:

- ❖ **None** is a special constant in Python that represents the absence of a value or a null value.
- ❖ **X=[]** - Here X is an empty list, and the list exists in memory. It can store the elements
- ❖ **X=None**- Here, X has no value, and it represents the absence of data.

```
: x=[]  
print(id(x))  
2078029744000
```

```
: y=[]  
print(id(y))  
2078029747328
```

```
x=None  
print(id(x))  
140733623896192
```

```
y=None  
print(id(y))  
140733623896192
```

Always use `is` instead of `==`

```
x=None  
if x==None:  
    print("hi")  
hi
```

```
: x=None  
if x is None:  
    print("hi")  
hi
```

For None, Python recommends an identity check, no value check

Data types: Sequence

List:

A list is an ordered sequence of items. It is one of the most used data types in Python and is very flexible. All the items in a list do not need to be of the same type.

How to create the list:

empty list

```
mylist = []
```

list of integers

```
mylist = [1, 2, 3]
```

list with mixed datatypes

```
mylist = [1, "Hello", 3.4]
```

nested list

```
mylist = ["mouse", [8, 4, 6], ['a']]
```

Data types: Sequence

User defined list:

```
# User inputs numbers separated by spaces
user_input = input("Enter numbers separated by space: ")

# Split the string and convert each to integer
numbers = list(map(int, user_input.split()))

print("You entered:", numbers)
```

Enter numbers separated by space: 2 3 4 5 6 7
You entered: [2, 3, 4, 5, 6, 7]

```
# User inputs numbers separated by spaces
user_input = input("Enter numbers separated by space: ")

# Split the string and convert each to integer
numbers = list(map(str, user_input.split()))

print("You entered:", numbers)
```

Enter numbers separated by space: ankit vijay vargiya matlab python
You entered: ['ankit', 'vijay', 'vargiya', 'matlab', 'python']

```
# Ask the user how many elements they want to enter
n = int(input("How many numbers do you want to enter? "))

# User inputs numbers separated by spaces
user_input = input(f"Enter {n} numbers separated by space: ")

# Split the string and convert each to integer
numbers = list(map(int, user_input.split()))

# Optional: Check if user entered exactly 'n' numbers
if len(numbers) != n:
    print(f"Warning: You entered {len(numbers)} numbers instead of {n}.")

print("You entered:", numbers)
```

How many numbers do you want to enter? 3
Enter 3 numbers separated by space: 1 2 3
You entered: [1, 2, 3]

Exercise

Write a Python program that builds a list where each element can have a different data type (int, float, or string).

Program Requirements:

- Ask the user how many elements they want to add to the list.
- For each element, prompt the user to:
 - Specify the data type (int, float, or str).
 - Enter the value.
- Convert the value to the specified data type.
- Append the converted value to the list.
- Finally, display the complete list.

```
n = int(input("Enter how many elements you want: "))

elements = []

for i in range(n):
    datatype = input(f"Enter datatype for element {i+1} (int / float / str): ")
    value = input(f"Enter value for element {i+1}: ")

    # Convert value based on datatype
    if datatype == "int":
        value = int(value)
    elif datatype == "float":
        value = float(value)
    elif datatype == "str":
        value = str(value)
    else:
        print("Invalid datatype entered. Treating as string by default.")
        value = str(value)

    elements.append(value)

print("Final List:", elements)
```

```
Enter how many elements you want: 3
Enter datatype for element 1 (int / float / str): str
Enter value for element 1: ankit
Enter datatype for element 2 (int / float / str): int
Enter value for element 2: 3
Enter datatype for element 3 (int / float / str): float
Enter value for element 3: 3.4
Final List: ['ankit', 3, 3.4]
```

Data types: Sequence

Accessing List Element:

```
mylist = [1, 2 , 3, 4]
```

```
print(mylist[0])
```

1

```
mylist = [1, "Hello", 3.4]
```

```
print(mylist[1])
```

Hello

```
print(mylist[1][1])
```

e

```
: mylist = ["mouse", [8, 4, 6], ['a']]
```

```
: print(mylist[1][2])
```

6

Data types: Sequence

Slicing Operator for Python List:

```
mylist=['a','b','c','d','e','f','g','h']
```

```
# print element 3rd to 5th  
print(mylist[2:5])
```

```
['c', 'd', 'e']
```

```
# print element beginning to 4th  
print(mylist[:4])
```

```
['a', 'b', 'c', 'd']
```

```
# print element 4th to end  
print(mylist[3:])
```

```
['d', 'e', 'f', 'g', 'h']
```

```
print(mylist[:-4])
```

```
['a', 'b', 'c', 'd']
```

```
print(mylist[-4:])
```

```
['e', 'f', 'g', 'h']
```

Data types: Sequence

Tuple:

- A tuple is similar to a list. The difference between the two is that we cannot change the elements of a tuple once it is assigned, whereas in a list, elements can be changed.
- Advantages of Tuple over List
 - Since tuples are immutable, iterating through a tuple is faster than with a list. So there is a slight performance boost.

Data types: Sequence

Creating a Tuple:

```
: # Empty Tuple  
my_tuple=()  
print(my_tuple)
```

`()`

```
: # Tuple having integer  
my_tuple=(1,2,3)  
print(my_tuple)
```

`(1, 2, 3)`

```
: # Tuple with mixed datatypes  
my_tuple= (1, "Hello" , 3.5)  
print(my_tuple)
```

`(1, 'Hello', 3.5)`

```
: # Nested Tuple  
my_tuple= ("Hello" , [1,2,3,4] , (1,2,3))  
print(my_tuple)
```

`('Hello', [1, 2, 3, 4], (1, 2, 3))`

Data types: Sequence

Assessing element in a Tuple:

```
: my_tuple=('a','b','c','d','e','f','g','h')
```

```
: print(my_tuple[0])
```

a

```
: print(my_tuple[5])
```

f

```
: print(my_tuple[8])
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-29-82cd0f8a39f9> in <module>  
----> 1 print(my_tuple[8])
```

IndexError: tuple index out of range

```
: print(my_tuple[1:3])
```

('b', 'c')

Data types: Sequence

Changing in a Tuple:

- ❖ Tuples are immutable, meaning you cannot update or change the values of tuple elements. You can take portions of existing tuples to create new tuples.

```
my_tuple1=(1,2,3)
my_tuple1[0]="hello"
print(my_tuple1)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-37-cc49132ecbe0> in <module>
      1 my_tuple1=(1,2,3)
----> 2 my_tuple1[0]="hello"
      3 print(my_tuple1)
```

```
TypeError: 'tuple' object does not support item assignment
```

```
my_list1=[1,2,3]
my_list1[0]="hello"
print(my_list1)
```

```
['hello', 2, 3]
```

Built-in Methods

Add an element to a List:

1. append

```
In [27]: x= [1,'Shyam',25]
```

```
In [28]: print(x)
```

```
[1, 'Shyam', 25]
```

```
In [29]: x.append("Male")
```

```
In [30]: print(x)
```

```
[1, 'Shyam', 25, 'Male']
```

```
In [1]: x= [1,'Shyam',25]
```

```
In [2]: print(x)
```

```
[1, 'Shyam', 25]
```

```
In [3]: y=['Male', 25000]
```

```
In [4]: print(y)
```

```
['Male', 25000]
```

```
In [5]: x.append(y)  
print(x)
```

```
[1, 'Shyam', 25, ['Male', 25000]]
```

Add an element to a Tuple:

```
: x=(1,2.3,"Ankit")  
x.append(5)
```

```
-----  
AttributeError                                Traceback (most recent call last)  
Cell In[1], line 2  
      1 x=(1,2.3,"Ankit")  
----> 2 x.append(5)  
  
AttributeError: 'tuple' object has no attribute 'append'
```

Built-in Methods

Add an element to a Python List:

2. Insert

```
In [1]: x= [1, 'Shyam',25]
```

```
In [2]: print(x)
```

```
[1, 'Shyam', 25]
```

```
In [3]: y=['Male', 25000]
```

```
In [4]: print(y)
```

```
['Male', 25000]
```

```
In [5]: x.insert(3,y)
```

```
In [6]: print(x)
```

```
[1, 'Shyam', 25, ['Male', 25000]]
```

3. extend

```
In [1]: x= [1, 'Shyam',25]
```

```
In [2]: print(x)
```

```
[1, 'Shyam', 25]
```

```
In [3]: y=['Male', 25000]
```

```
In [4]: print(y)
```

```
['Male', 25000]
```

```
In [5]: x.extend(y)
```

```
In [6]: print(x)
```

```
[1, 'Shyam', 25, 'Male', 25000]
```

Error with a Tuple

Built-in Methods

4. + operator

List

```
In [1]: x= [1,'Shyam',25]
In [2]: print(x)
        [1, 'Shyam', 25]
In [3]: y=['Male', 25000]
In [4]: print(y)
        ['Male', 25000]
In [5]: print(x+y)
        [1, 'Shyam', 25, 'Male', 25000]
```

Tuple

```
x=(1,'shyam',25)
y=('Male',25000)
print(x+y)
```

```
x=(1,'shyam',25)
y=('Male',25000)
print(x+y)
(1, 'shyam', 25, 'Male', 25000)
```

Built-in Methods

Add multiple elements at a specific location in List:

Using for loop

```
a=[1,2,3,4,5]
b=[6,7]
pos=2
for i in b:
    a.insert(pos,i)
    pos=pos+1
print(a)
```

[1, 2, 6, 7, 3, 4, 5]

With slicing

```
: a=[1,2,3,4,5]
   b=[6,7]

   a[2:2]=b
   print(a)
```

[1, 2, 6, 7, 3, 4, 5]

```
a=[1,2,3,4,5]
b=[6,7]

a[2:4]=b
print(a)
```

[1, 2, 6, 7, 5]

```
: a=[1,2,3,4,5]
   b=[6,7]

   a[2:3]=b
   print(a)
```

[1, 2, 6, 7, 4, 5]

```
: a=[1,2,3,4,5]
   b=[6,7]

   a[2:-1]=b
   print(a)
```

[1, 2, 6, 7, 5]

Built-in Methods

Delete Element

List

```
In [9]: mylist=['a','b','c','d','e','f','g','h']
```

```
In [10]: #del one value  
del mylist[2]
```

```
In [11]: print(mylist)  
  
['a', 'b', 'd', 'e', 'f', 'g', 'h']
```

```
In [9]: mylist=['a','b','c','d','e','f','g','h']
```

```
In [12]: #del multiple value  
del mylist[2:4]
```

```
In [13]: print(mylist)  
  
['a', 'b', 'f', 'g', 'h']
```

Tuple

```
: a=(1,2,3)  
del a[0]  
a
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[9], line 2  
      1 a=(1,2,3)  
----> 2 del a[0]  
      3 a  
  
TypeError: 'tuple' object doesn't support item deletion
```

```
del a
```

```
a
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[11], line 1  
----> 1 a  
  
NameError: name 'a' is not defined
```

Built-in Methods

Mutability inside containers:

```
a=("Hello", [1,2,3,4,5], (1,2,3))
```

```
a[1]=5
```

```
a[2][1]=4
```

```
a[1][1]=8
```

```
a[1]=5
a
```

```
-----
TypeError
Cell In[2], line 1
----> 1 a[1]=5
      2 a
```

```
a[2][1]=4
```

```
-----
TypeError
Cell In[3], line 1
----> 1 a[2][1]=4
```

```
a[1][1]=8
a
```

```
('Hello', [1, 8, 3, 4, 5], (1, 2, 3))
```

```
a=("Hello", [1,2,3,4,5], (1,2,3))
a[1].append(9)
a
```

```
a=("Hello", [1,2,3,4,5], (1,2,3))
a[1].append(9)
a
```

```
('Hello', [1, 2, 3, 4, 5, 9], (1, 2, 3))
```

```
: del a[1][1]
```

```
del a[0][1]
```

```
a[2].append(9)
```

```
a[2].append(9)
```

```
-----
AttributeError
Cell In[8], line 1
----> 1 a[2].append(9)
```

```
a=("Hello", [1,2,3,4,5], (1,2,3))
del a[1][1]
a
```

```
('Hello', [1, 3, 4, 5], (1, 2, 3))
```

```
-----
TypeError
Cell In[12], line 1
----> 1 del a[0][1]
```